



Defensa Individual EV3 — Guía de Estudio

DSY1103 Desarrollo FullStack I · Proyecto Estacionamiento Inteligente · Grupo 7 (Sol León · Catalina Aguirre)



Defensa: miércoles 02/07/2026 · 15 min por estudiante · sin IA, sin internet (salvo dependencias Maven)

Contenido

1. Si solo estudias 3 cosas, que sean estas
2. Cómo funciona la evaluación de mañana
3. TESTING — la sección más importante (20%)
4. Overall del proyecto en 10 minutos
5. Lo que el profesor dijo que faltó en EV2
6. Levantar el sistema sola (10% + 6%)
7. Banco de preguntas con respuestas modelo
8. Checklist de la noche anterior

0. Si solo estudias 3 cosas, que sean estas

Dónde están los puntos (dimensión Defensa = 60% de la rúbrica EV3):

1. **13%** **Escribir una prueba unitaria NUEVA en vivo** (IE 3.1.3). Es el indicador individual más pesado de toda la rúbrica. Practica la plantilla de la sección 2.6 hasta poder escribirla de memoria.
2. **10%** **Levantar y configurar el ecosistema sin ayuda** (IE 3.3.7): orden de arranque, puertos, resolver un error en vivo. Sección 5.
3. **7%** **Explicar los tests existentes** (IE 3.1.2): qué mockea, qué assert usa, qué regla de negocio valida. Sección 2.

Solo esos tres indicadores suman **30 de los 60 puntos** de la defensa. El resto (código, DTOs, aporte personal, Swagger, YAML, despliegue) suma los otros 30.

Actualización (Catalina, 01/07/2026):

el profesor confirmó que **no** va a pedir escribir/integrar una prueba unitaria nueva en vivo durante la defensa, aunque IE 3.1.3 siga listado en la rúbrica oficial con 13%. La receta de la sección 2.6 sigue sirviendo como preparación de respaldo, pero el foco real de "TESTING" en la defensa es **explicar** los tests que ya existen (IE 3.1.2, sección 2.2-2.5), no demostrar autoría en vivo.

Además, el profesor ya te dijo por escrito qué le faltó a Sol en EV2 (65.4/100):

GlobalExceptionHandler y códigos HTTP, aporte personal con evidencia, un escenario de error concreto, y una entidad formal con PK y restricciones. Todo eso está resuelto y listo para decir en la sección 4.

1. Cómo funciona la evaluación de mañana

Logística (de la rúbrica oficial y anuncios del profesor)

- La defensa es **individual**: la nota depende solo de lo que cada una demuestre, aunque el proyecto grupal esté perfecto.
- **Máximo 15 minutos** por estudiante. El orden se asigna al azar.
- **Prohibido**: IA, herramientas automáticas de código, ayuda externa. Internet solo para dependencias Maven.
- El profesor prepara **preguntas personalizadas a partir del repo** (lo descargó y auditó el 30/06).
- Puedes pedir que te repitan la pregunta y puedes **anotar los pasos** antes de ejecutarlos. Se evalúa tanto lo que haces como lo que explicas.
- Regla dura: si un recurso entregado no está implementado de verdad, todos los ítems asociados se califican con 0.

La rúbrica de la dimensión Defensa (60%)

Indicador	Qué evalúa	Peso	Para el 100% necesitas...
IE 3.1.3	Integrar una prueba unitaria nueva en tiempo acotado	13%	Test funcional con Given–When–Then, mocks precisos, asserts coherentes; compila y pasa; seguridad al modificar código en vivo.
IE 3.3.7	Configurar y ejecutar los microservicios sin apoyo	10%	Levantar servicios, ajustar puertos/perfiles/rutas del Gateway sin asistencia; diagnosticar y corregir problemas rápido.
IE 3.1.2	Explicar las pruebas unitarias existentes	7%	Estructura del test, función evaluada, propósito de cada assert, uso de mocks, qué regla de negocio se valida y por qué esos casos.
IE 3.3.6	Describir el proceso de despliegue	6%	Plataforma (Docker/nativo), variables, puertos, logs, arranque, e interpretar errores de despliegue y cómo se resolvieron.
IE 2.2.2	Explicar el código y justificar decisiones	5%	Relaciones, lógica, responsabilidades por capa, impactos de las decisiones de diseño.
IE 2.4.2	Consistencia de datos e interoperabilidad (endpoints, DTOs, consumo remoto)	5%	Explicar el flujo remoto exacto, diseño de endpoints y DTOs, validaciones y coherencia entre servicios.
IE 2.5.3	Aporte personal (commits, tareas, participación)	5%	Claridad total sobre tu trabajo con commits, archivos y roles concretos. <i>(Sol sacó 30% aquí en EV2 — ver sección 4.)</i>
IE 3.2.2	Explicar la documentación Swagger	5%	Recorrer la UI: rutas, parámetros, modelos, request bodies, códigos de respuesta; cómo consumir cada endpoint.
IE 3.3.5	Explicar la configuración YAML	4%	Propiedades clave, función de cada sección, local vs remoto, cómo el YAML afecta el despliegue y la comunicación.

Los otros 40 puntos (dimensión Entrega grupal) ya se jugaron con la entrega del 29/06 —
nota grupal 7.0.

2. TESTING

IE 3.1.2 + 3.1.3 = 20%

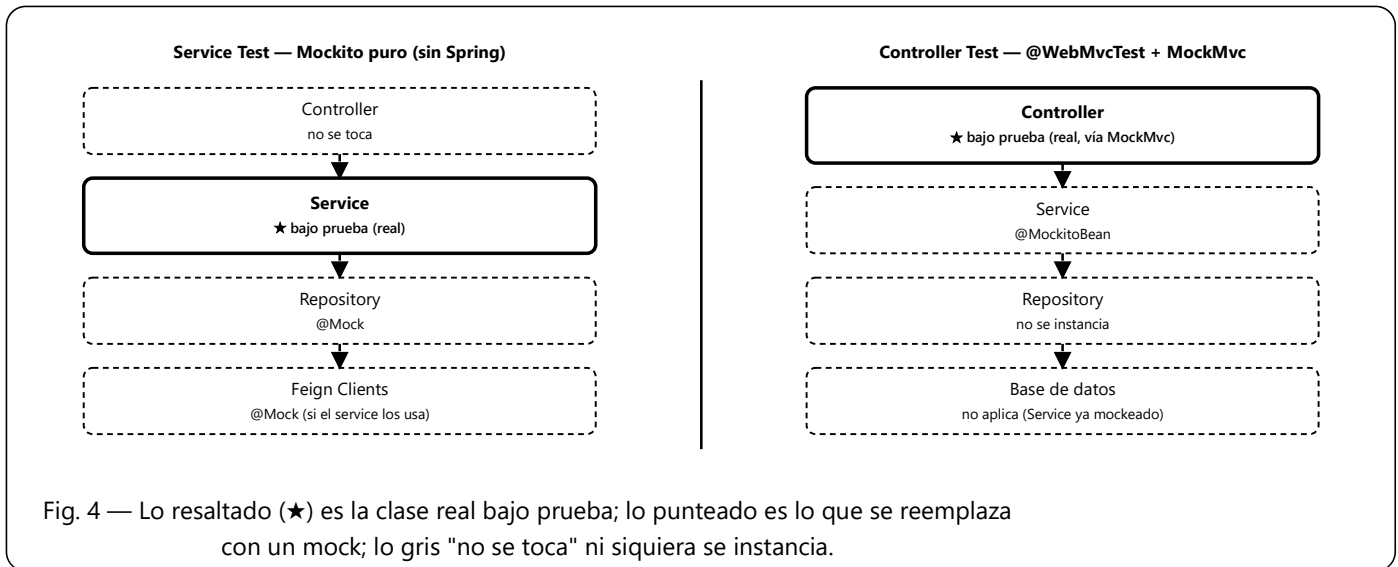
Basado en la guía oficial "3.1 Análisis de testing en Biblioteca y transición a microservicios" (Unidad 3), pero con ejemplos **del propio proyecto** — que es de donde el profesor sacará las preguntas.

2.1 Los conceptos que la guía exige dominar

Concepto	Qué es / qué hace
Prueba unitaria	Prueba automatizada de una unidad pequeña de código (un método del Service, una regla de negocio, una respuesta de endpoint) de forma aislada , sin BD real ni servicios externos.
Mock	Objeto simulado que reemplaza una dependencia real (Repository, Feign Client). Permite probar el Service sin MySQL y sin que los otros microservicios estén corriendo.
@Mock vs @MockBean	@Mock (Mockito puro) crea el simulado en un test aislado sin levantar Spring — es lo que usan nuestros Service tests. @MockBean/@MockitoBean reemplaza un bean dentro del contexto de Spring — lo usamos en el Controller test con @WebMvcTest .
@InjectMocks	Crea la instancia real del Service bajo prueba e inyecta en ella todos los @Mock declarados.
when(...).thenReturn(...)	Programa el mock: "cuando alguien llame a findById(1L) , devuelve este objeto simulado".
verify(...)	Comprueba que un método del mock fue llamado (o no: verify(repo, never()).save(any())). Es clave en métodos que no retornan nada, como eliminaciones o llamadas a otros servicios.
Asserts	La guía usa JUnit (assertEquals , assertNotNull). Nuestro proyecto usa AssertJ : assertThat(x).isEqualTo(y) , assertThat(lista).hasSize(1) , assertThatThrownBy(...).isInstanceOf(...) . Cumplen el mismo rol: verificar el resultado esperado. Si el profesor pide JUnit clásico, di la equivalencia.
MockMvc	Simula peticiones HTTP contra el Controller sin levantar un servidor real : mockMvc.perform(get("/api/espacios")).andExpect(status().isOk()) .
jsonPath()	Valida el contenido del JSON de respuesta: jsonPath("\$.numero").value("A-01") .
ObjectMapper	Convierte objetos Java a JSON para el body de POST/PUT: .content(objectMapper.writeValueAsString(dto)) .
Given-When-Then	Estructura del test: <i>Given/Arrange</i> (preparar datos y mocks) → <i>When/Act</i> (ejecutar el método) → <i>Then/Assert</i> (verificar). En nuestro código los comentarios dicen // Arrange , // Act , // Assert — es la misma convención (AAA); dilo explícitamente si te preguntan por Given-When-Then.

Diferencia Service test vs Controller test (pregunta segura):

El test de **Service** valida *lógica de negocio*: reglas, cálculos, excepciones; se mockea el Repository (y los Feign Clients). El test de **Controller** valida el *contrato HTTP*: ruta, método, código de respuesta y JSON; se mockea el Service y se usa MockMvc. Un test de Controller NO valida la base de datos.



2.2 Anatomía de un test real del proyecto, línea a línea

De `ms-accesos/src/test/java/.../service/AccesoServiceTest.java` (este archivo existe tal cual en el repo):

```
@ExtendWith(MockitoExtension.class) // activa Mockito en JUnit 5, sin levantar Spring
class AccesoServiceTest {

    @Mock private AccesoRepository accesoRepository; // simula la BD
    @Mock private EspacioClient espacioClient;      // simula el Feign hacia ms-espacios
    @Mock private ReservaClient reservaClient;      // simula el Feign hacia ms-reservas

    @InjectMocks
    private AccesoService accesoService;            // la clase REAL bajo prueba

    @Test
    @DisplayName("registrarEntrada debe crear el acceso y marcar el espacio no disponible")
    void registrarEntrada_valido_debeCrearAccesoYBloquearEspacio() {
        // Arrange (Given): la reserva remota está CONFIRMADA y no hay acceso previo
        when(reservaClient.findById(1L)).thenReturn(reservaConfirmada);
        when(accesoRepository.findByIdReserva(1L)).thenReturn(Optional.empty());
        when(accesoRepository.save(any(Acceso.class))).thenAnswer(inv -> {
            Acceso a = inv.getArgument(0);
            a.setId(10L); // simula el ID que asignaría MySQL
            return a;
        });

        // Act (When): se ejecuta el método real del Service
        AccesoResponseDTO resultado = accesoService.registrarEntrada(createDTO);

        // Assert (Then): estado correcto y efecto lateral verificado
        assertThat(resultado.getId()).isEqualTo(10L);
        assertThat(resultado.getEstado()).isEqualTo("ACTIVO");
        verify(espacioClient).updateDisponibilidad(1L, false); // ¡bloqueó el espacio!
    }
}
```

Qué regla de negocio valida: al registrar una entrada con reserva CONFIRMADA y sin acceso duplicado, se crea el Acceso en estado ACTIVO y *además* se llama a ms-espacios para marcar el espacio como no disponible. El **verify()** final es lo que prueba la comunicación entre microservicios sin necesitar que ms-espacios exista.

2.3 La tabla de la guía (sección 7.5), rellena con nuestro test

Si el profesor te muestra cualquier test y pregunta "explícalo", responde estas 9 preguntas en orden:

Pregunta de la guía	Respuesta para registrarEntrada_reservaNoConfirmada_debeLanzarConflict
¿Qué clase se está probando?	AccesoService (ms-accesos)
¿Qué método se está probando?	registrarEntrada(AccesoCreateDTO)
¿Qué dependencia se simula?	AccesoRepository, EspacioClient y ReservaClient (¡en microservicios se mockean también los Feign Clients!)
¿Qué datos se preparan?	Una ReservaResponseDTO con estado "PENDIENTE" (no CONFIRMADA)
¿Qué método se ejecuta?	accesoService.registrarEntrada(createDTO) — el método real
¿Qué assert se utiliza?	assertThatThrownBy(...).isInstanceOf(ConflictException.class)
¿Qué resultado se espera?	Que lance ConflictException y que no se guarde nada: verify(accesoRepository, never()).save(any())
¿Usa la base de datos real?	No. Todo está mockeado; no se conecta a MySQL ni a otros servicios.
¿Qué podría fallar?	Mock mal configurado (ej. when con otro ID), mensaje esperado distinto, o que el Service cambie la regla y ya no lance Conflict.

2.4 Controller test con MockMvc (el único del repo: ms-espacios)

De **ms-espacios/src/test/java/ ... /controller/EspacioControllerTest.java**.

Importante conocerlo porque la guía dedica media sección a MockMvc:

```

@WebMvcTest(EspacioController.class)      // levanta SOLO la capa web de este controller
@AutoConfigureMockMvc(addFilters = false) // desactiva filtros de seguridad en el test
class EspacioControllerTest {

    @Autowired private MockMvc mockMvc;
    @Autowired private ObjectMapper objectMapper;

    @MockitoBean                          // mock del Service dentro del contexto Spring
    private EspacioService espacioService;

    @Test
    void getById_inexistente_debeRetornar404() throws Exception {
        // Arrange: el service simulado lanza la excepción de "no encontrado"
        when(espacioService.findById(99L))
            .thenReturn(new ResourceNotFoundException("Espacio no encontrado con id: 99"));

        // Act + Assert: petición simulada y validación del contrato HTTP
        mockMvc.perform(get("/api/espacios/99"))
            .andExpect(status().isNotFound())                // 404
            .andExpect(jsonPath("$.error").value("NOT_FOUND")) // body del handler
            .andExpect(jsonPath("$.mensaje").value("Espacio no encontrado con id: 99"));
    }

    @Test
    void create_valido_debeRetornar201() throws Exception {
        when(espacioService.create(any(EspacioCreateDTO.class))).thenReturn(espacioDTO());

        mockMvc.perform(post("/api/espacios")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(dto))) // objeto → JSON
            .andExpect(status().isCreated());                // 201
    }
}

```

Fíjate en la cadena completa que valida el test del 404:

Controller recibe GET → Service mockeado lanza **ResourceNotFoundException** → el **GlobalExceptionHandler** (@RestControllerAdvice) la captura y responde **404** con body {**error: "NOT_FOUND", mensaje, timestamp**}. Este test demuestra en una sola pieza: MockMvc + manejo de excepciones + códigos HTTP — exactamente lo que le faltó a Sol en EV2 (IE 2.3.4).

El test **create_invalido_debeRetornar400** además valida Bean Validation: envía un DTO sin campos obligatorios y espera **400** con **jsonPath("\$.campos.numero").exists()** — el mapa de campos que arma el handler ante **MethodArgumentNotValidException**.

2.5 Lo que cambia en microservicios (transición de la guía)

En el monolito Biblioteca se mockeaba solo el Repository. En microservicios **también se mockea la comunicación externa**. Nuestro mejor ejemplo es **ReservaServiceTest** (ms-reservas), que mockea 1 repository + 3 Feign Clients:


```

@Mock private ReservaRepository reservasRepository;
@Mock private ClienteClient clienteClient;    // → user-service
@Mock private VehiculoClient vehiculoClient;  // → ms-vehiculos
@Mock private EspacioClient espacioClient;    // → ms-espacios

// Arrange típico: los 3 servicios remotos responden OK
when(clienteClient.findById(1L)).thenReturn(new ClienteResponseDTO(1L, true));
when(vehiculoClient.findById(1L)).thenReturn(new VehiculoResponseDTO(1L, 1L, true));
when(espacioClient.findById(1L)).thenReturn(new EspacioResponseDTO(1L, "A-01", "Norte", 1, 1L, true, true));

```

Por qué se simula la comunicación externa (pregunta 7 de la actividad de reflexión de la guía): porque una prueba unitaria debe ser determinista y aislada — si dependiera de que user-service esté corriendo, el test fallaría por razones ajenas al código probado (red, orden de arranque, datos). El mock convierte "el otro servicio" en una respuesta controlada.

Detalle avanzado que suma puntos: **PagoServiceTest** usa

ReflectionTestUtils.invokeMethod(...) para probar los métodos **privados** de la fórmula (**calcularMontoBase**, **calcularMontoFinal**) con valores exactos de **BigDecimal** — p. ej. valida que $\$1000/h \times \text{horario } 2.0 \times \text{vehículo } 1.5 \times \text{espacio } 1.2 \times 1.5h = \mathbf{5400.00}$, y la cascada de descuentos 10%/20%/5% sobre $\$10.000 = \mathbf{6840.00}$.

2.6 ★ Receta para escribir un test NUEVO en vivo (IE 3.1.3 — 13%)

Este es el momento de mayor puntaje de toda la defensa.

El profesor pedirá algo como "agrega una prueba que valide X regla". Memoriza la plantilla y el procedimiento. Puedes anotar los pasos en papel antes de escribir.

Procedimiento (di lo que haces en voz alta):

1. **Ubicar el test existente** del service correspondiente en **src/test/java/ ... /service/** — nunca partir de un archivo vacío: copia la estructura de un **@Test** vecino.
2. **Identificar la regla**: ¿qué método del Service? ¿caso feliz o caso de error?
3. **Arrange**: programar con **when(...)** SOLO los mocks que ese método usa en ese camino (Mockito en modo strict falla con stubs innecesarios: **UnnecessaryStubbingException**).
4. **Act**: llamar al método real.
5. **Assert**: para caso feliz **assertThat(...) + verify(...)**; para caso de error **assertThatThrownBy(...) + verify(repo, never()).save(any())**.
6. **Ejecutar**: **.\mvnw.cmd test -Dtest=AccesoServiceTest** desde la carpeta del servicio, y mostrar el verde.

Plantilla de memoria (caso de error, el más pedido):

```
@Test
@DisplayName("descripción en español de la regla que valida")
void metodo_condicion_resultadoEsperado() {
    // Arrange (Given)
    when(dependenciaMock.metodo(argumento)).thenReturn(valorPreparado);

    // Act + Assert (When + Then)
    assertThatThrownBy(() → serviceReal.metodoBajoPrueba(dto))
        .assertInstanceOf(ConflictException.class)
        .hasMessageContaining("parte del mensaje");
    verify(repositoryMock, never()).save(any());
}
```

Ejercicio resuelto 1 — "valida que no se pueda registrar salida dos veces" (ya existe uno casi igual; así se escribiría desde cero):

```
@Test
@DisplayName("registrarSalida debe lanzar ConflictException si la salida ya fue registrada")
void registrarSalida_salidaYaRegistrada_debeLanzarConflict() {
    // Given: un acceso que YA tiene fechaHoraSalida
    accesoActivo.setFechaHoraSalida(LocalDateTime.now());
    when(accesoRepository.findById(1L)).thenReturn(Optional.of(accesoActivo));

    // When + Then
    assertThatThrownBy(() → accesoService.registrarSalida(1L))
        .assertInstanceOf(ConflictException.class)
        .hasMessageContaining("Salida ya registrada");
}
```

Línea a línea: preparo el estado inválido (salida ya seteada) → programo el repo para devolver ese acceso → ejecuto y afirmo que lanza **ConflictException**. No necesito mockear **espacioClient** porque el método falla antes de llegar a esa llamada.

Ejercicio resuelto 2 — caso feliz con verify de Feign: "al registrar salida se libera el espacio y se finaliza la reserva":

```
@Test
void registrarSalida_valido_liberaEspacioYFinalizaReserva() {
    // Given: acceso ACTIVO con entrada hace 1 hora (del setUp)
    when(accesoRepository.findById(1L)).thenReturn(Optional.of(accesoActivo));
    when(accesoRepository.save(any(Acceso.class))).thenReturn(inv -> inv.getArgument(0));

    // When
    AccesoResponseDTO resultado = accesoService.registrarSalida(1L);

    // Then: minutos calculados, estado COMPLETADO, y efectos remotos verificados
    assertThat(resultado.getEstado()).isEqualTo("COMPLETADO");
    assertThat(resultado.getMinutos()).isGreaterThan(0L);
    verify(espacioClient).updateDisponibilidad(1L, true);    // libera el espacio
    verify(reservaClient).finalizarReserva(1L);              // cierra la reserva
}
```

Ejercicio resuelto 3 — si pide un test en otro servicio (patrón findById inexistente, sirve para CUALQUIER service del repo):

```
@Test
@DisplayName("findById debe lanzar NotFoundException cuando el id no existe")
void findById_inexistente_lanzaNotFound() {
    when(xxxRepository.findById(99L)).thenReturn(Optional.empty());

    assertThatThrownBy(() -> xxxService.findById(99L))
        .isInstanceOf(NotFoundException.class);
}
```

Ojo con el nombre de la excepción por servicio:

ms-accesos y ms-reservas usan **NotFoundException** / **ConflictException**; ms-espacios y ms-pagos usan **ResourceNotFoundException**; ms-pagos además tiene **BusinessException** (ej.: "Ya existe un cobro" para el UNIQUE 1:1 con acceso). Antes de escribir, mira los imports del test vecino.

2.7 Errores comunes al ejecutar pruebas (sección 11 de la guía) y cómo se ven aquí

Error de la guía	Síntoma	Cómo lo evitamos en este proyecto
Conexión a BD real	Communications link failure, Access denied for user 'root', Unknown database	Los Service tests con <code>@ExtendWith(MockitoExtension.class)</code> ni siquiera levantan Spring. Los de integración usan H2 en memoria vía <code>src/test/resources/application-test.properties</code> .
Perfil incorrecto	El test usa la config de desarrollo	<code>@ActiveProfiles("test")</code> + el properties de test con <code>ddl-auto=create-drop</code> y <code>eureka.client.enabled=false</code> / <code>spring.cloud.discovery.enabled=false</code> (para que el test no intente registrarse en Eureka).

Ruta incorrecta	404 Not Found en MockMvc	La ruta del test debe calzar EXACTA con el @RequestMapping del controller (ej. / api/espacios).
Código HTTP distinto	Test espera isOk() pero el endpoint devuelve 201	Nuestro POST de espacios devuelve 201 Created → el test usa status().isCreated() , no isOk() .
Campo JSON distinto	jsonPath no encuentra el campo	Validar contra el DTO real (ej. \${[0].numero, \$.tipoEspacio.nombre}).
Falta un mock	No qualifying bean of type EspacioService	En @WebMvcTest hay que declarar @MockitoBean del Service, si no Spring no puede construir el Controller.

Extra frecuente en Mockito moderno: **UnnecessaryStubbingException** — programaste un **when(...)** que el camino probado nunca usa. Solución: borrar ese stub o moverlo solo a los tests que lo necesitan.

2.8 Comandos de testing (memoriza)

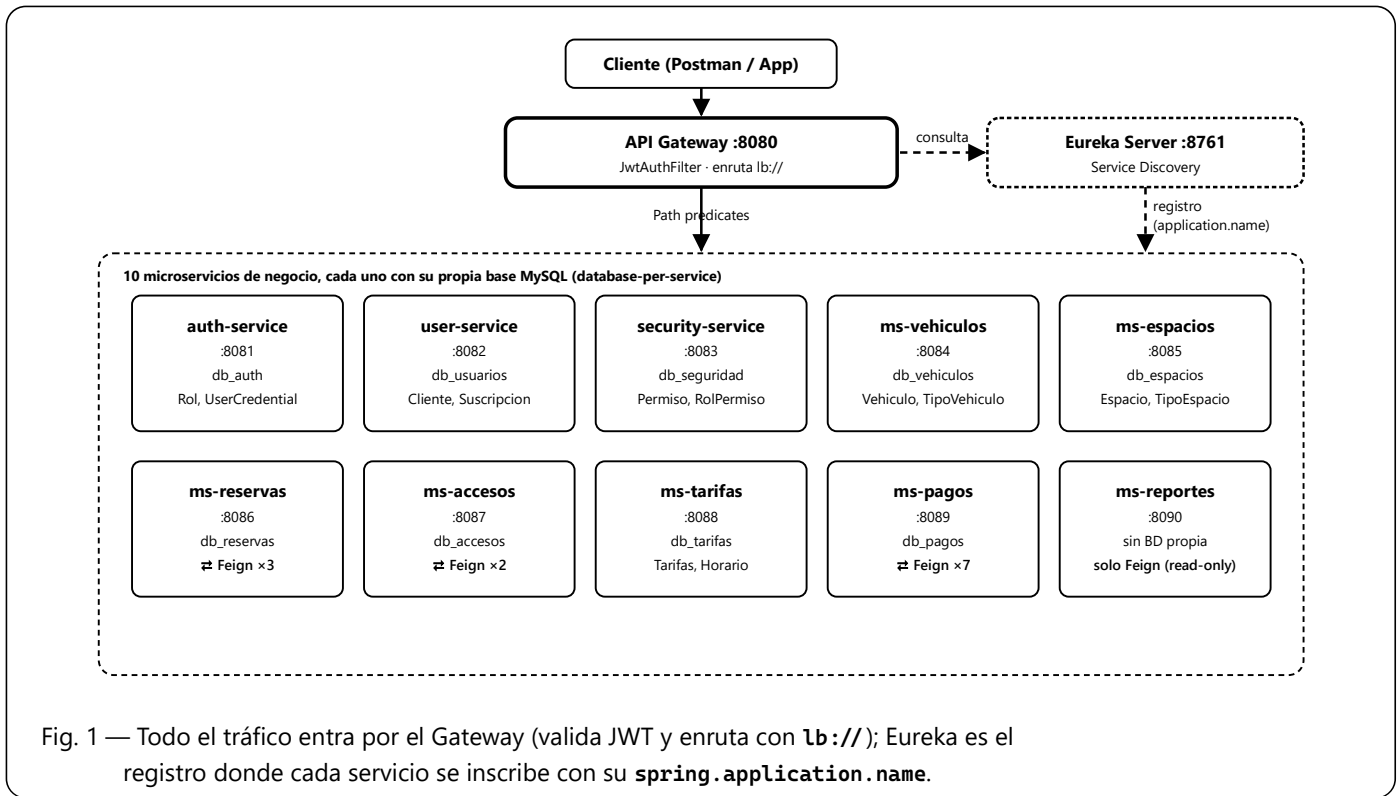
```
# desde la carpeta del servicio (mvn NO está global - usar el wrapper)
cd ms-accesos
.\mvnw.cmd test                # toda la suite del servicio
.\mvnw.cmd test -Dtest=AccesoServiceTest # una clase específica
.\mvnw.cmd verify              # tests + reporte JaCoCo → target/site/jacoco/index.html
.\mvnw.cmd clean install       # compila + ejecuta tests (lo que menciona el README)
```

3. Overall del proyecto en 10 minutos

IE 2.2.2 + 2.4.2 + 3.3.5

3.1 Arquitectura: 12 servicios

Servicio	Puerto	BD	Responsabilidad
eureka-server	8761	—	Registro/descubrimiento de servicios (Service Discovery)
api-gateway	8080	—	Punto único de entrada; enruta con <code>lb://</code> y valida JWT (<code>JwtAuthFilter</code>)
auth-service	8081	db_auth	Login/registro; emite JWT; BCrypt fuerza 12; entidades <code>RoI</code> , <code>UserCredential</code> (login por email)
user-service	8082	db_usuarios	<code>Cliente</code> , <code>TipoCliente</code> , <code>Suscripcion</code> , <code>ClienteSuscripcion</code>
security-service	8083	db_seguridad	<code>Permiso</code> , <code>RoIPermiso</code>
ms-vehiculos	8084	db_vehiculos	<code>Vehiculo</code> (patente inmutable), <code>TipoVehiculo</code> (factor de cobro)
ms-espacios	8085	db_espacios	<code>Espacio</code> , <code>TipoEspacio</code> ; PUT <code>/disponibilidad</code> ; soft delete <code>activo=false</code>
ms-reservas	8086	db_reservas	<code>Reserva</code> + máquina de estados (<code>EstadoEnums</code>)
ms-accesos	8087	db_accesos	<code>Acceso</code> : entrada/salida, cálculo de minutos
ms-tarifas	8088	db_tarifas	<code>Tarifas</code> , <code>HorarioTarifas</code> (LABORAL / FIN_DE_SEMANA / FESTIVO); GET <code>/vigente</code>
ms-pagos	8089	db_pagos	<code>Cobro</code> (UNIQUE 1:1 con acceso), <code>Banco</code> , <code>MetodoPago</code> , <code>TipoTarjeta</code> ; fórmula <code>BigDecimal</code>
ms-reportes	8090	— (solo Feign)	Reportes no persistentes: ocupación, accesos por reserva, cobros por cliente



Stack: Spring Boot 3.5.14 · Java 21 · MySQL 8 · Spring Cloud 2025.0.2 (Gateway + Eureka + OpenFeign) · JWT 0.11.5 · springdoc-openapi 2.5.0 · JUnit 5 + Mockito + AssertJ + H2.

3.2 El YAML del Gateway (IE 3.3.5 — 4%; cítaló con confianza, es real)

```

server:
  port: 8080                # puerto único de entrada
spring:
  application:
    name: api-gateway        # nombre con el que se registra en Eureka
  cloud:
    gateway:
      discovery:
        locator:
          enabled: true       # descubre servicios desde Eureka
      routes:
        - id: ms-accesos
          uri: lb://ms-accesos      # lb = load-balanced: resuelve por Eureka
          predicates:
            - Path=/api/accesos/**    # predicado: qué requests entran a esta ruta
        - id: ms-pagos
          uri: lb://ms-pagos
          predicates:
            - Path=/api/cobros/**,/api/bancos/**,/api/metodos-pago/**,/api/tipo-tarjetas/**
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/  # dónde está Eureka
jwt:
  secret: ...                # clave Base64 ≥256 bits para validar tokens

```

Cómo explicarlo: `lb://ms-accesos` significa que el Gateway *no conoce IP ni puerto*: le pregunta a Eureka "¿dónde vive ms-accesos?" y balancea entre las instancias registradas. El **predicate Path** decide qué peticiones van a cada servicio. Además el Gateway aplica el **JwtAuthFilter**: toda ruta excepto `/api/auth/**` y Swagger exige un token válido. En el **test** los YAML/properties cambian a `H2 + eureka.client.enabled=false`; en **Docker**, `localhost` se reemplaza por nombres de servicio (`eureka-server`, `mysql-db`) — esa es la diferencia local vs remoto que pide la rúbrica.

3.3 Flujo de negocio completo: reserva → acceso → cobro → reporte



Fig. 2 — Flujo secuencial: cada etapa consulta la anterior por Feign; ninguna tiene acceso directo a la base de datos de otra.

1. **Reserva (ms-reservas): create()** valida por Feign que el cliente esté activo (user-service), el vehículo activo (ms-vehiculos) y el espacio activo+disponible (ms-espacios) → guarda en

- PENDIENTE.** Máquina de estados: PENDIENTE → CONFIRMADA (bloquea el espacio vía Feign) o CANCELADA (no toca el espacio, nunca se bloqueó); CONFIRMADA → FINALIZADA.
2. **Acceso (ms-accesos): registrarEntrada** valida la reserva CONFIRMADA (ReservaClient), evita duplicados, crea **Acceso** ACTIVO y marca el espacio no disponible (EspacioClient). **registrarSalida** calcula **minutos**, pasa a COMPLETADO, libera el espacio y finaliza la reserva. **idReserva** es nullable: se permite acceso sin reserva.
 3. **Cobro (ms-pagos):** con el acceso COMPLETADO, consulta por Feign: acceso, tarifa vigente, horario vigente, tipo de vehículo, tipo de espacio, cliente/suscripción y banco → aplica la fórmula (abajo). UNIQUE en **id_acceso_ref**: 1 cobro por acceso.
 4. **Reporte (ms-reportes):** sin BD propia; agrega datos vía Feign (ocupación de espacios, accesos por reserva, cobros por cliente). Solo lectura.

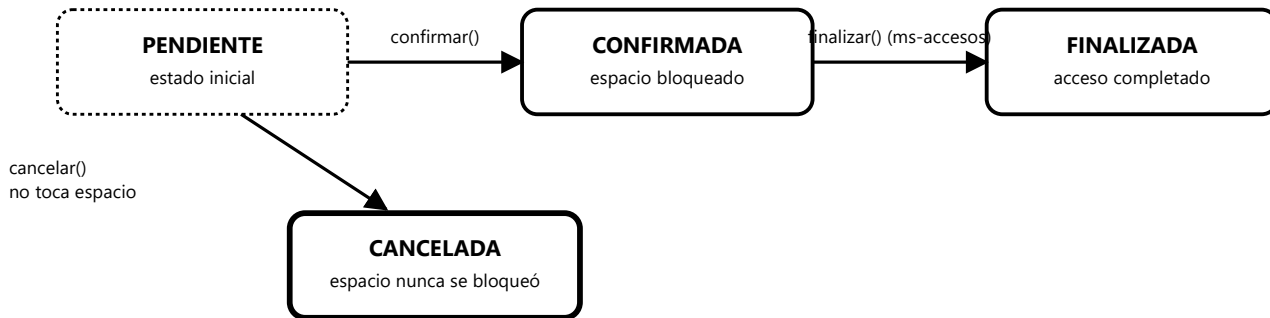


Fig. 3 — Desde PENDIENTE solo se puede confirmar o cancelar; desde CONFIRMADA solo se puede finalizar (vía ms-accesos al registrar la salida).

3.4 Fórmula de cobro (dila de memoria)

```

monto_base = precio_base_hora × multiplicador_horario
             × factor_tipo_vehiculo × factor_tipo_espacio
             × (minutos / 60)

monto_final = monto_base × (1 - desc_tipo_cliente/100)
                × (1 - desc_suscripcion/100)
                × (1 - desc_banco/100)      ← descuentos EN CASCADA, no sumados
  
```

Todo con **BigDecimal** y **RoundingMode.HALF_UP** (nunca **double** para dinero). Ejemplo verificado por test: base \$10.000 con descuentos 10%, 20% y 5% → $10000 \times 0.90 \times 0.80 \times 0.95 = \$6.840,00$.

3.5 Mapa Feign y referencias lógicas


```
ms-reservas → EspacioClient, VehiculoClient, ClienteClient
ms-accesos  → EspacioClient, ReservaClient
ms-pagos    → AccesoClient, TarifaClient, ClienteClient, VehiculoClient,
             EspacioClient, HorarioTarifaClient, TipoVehiculoClient
ms-reportes → AccesoClient, CobroClient, EspacioClient, VehiculoClient
```

- El **name** del **@FeignClient** debe calzar exacto con el **spring.application.name** del servicio destino — así Eureka lo resuelve.
- **Referencias lógicas:** entre bases NO hay FK físicas; se guardan **Long** planos (**@Column(name="id_espacio_ref")**). La integridad se garantiza *en la capa de servicio*, validando por Feign antes de persistir. Justificación: cada microservicio es dueño de su BD (database-per-service); una FK cruzada acoplaría los esquemas y rompería el despliegue independiente.
- **Solo @PutMapping, nunca @PatchMapping:** el **URLConnection** por defecto de Feign no soporta PATCH; todas las actualizaciones parciales (cancelar, confirmar, finalizar, disponibilidad, salida) son PUT.
- Feign devuelve los 404 remotos como **FeignException** (no como **null**) — por eso los services capturan y traducen la excepción.

3.6 Swagger/OpenAPI (IE 3.2.2 — 5%)

- Dependencia: **springdoc-openapi-starter-webmvc-ui** 2.5.0 (centralizada en el pom padre).
- UI por servicio en su puerto directo: **http://localhost:8085/swagger-ui/index.html** (ms-espacios), 8087 (accesos), 8089 (pagos)...
- Qué mostrar si te piden recorrerla: la ruta y método de un endpoint → parámetros de entrada → el **schema del DTO** del request body → los códigos de respuesta (200/201/400/404/409) → el ejemplo JSON. Y decir que la doc se genera desde el código (anotaciones + tipos), por eso coincide con el comportamiento real.

4. Lo que el profesor dijo que faltó en EV2 → respuestas listas

Sol: 65.4/100 en EV2. Estas 5 debilidades están escritas en la revisión del profesor y son sus focos favoritos. Catalina: te aplican igual — son las mismas trampas.

4.1 ❌ "No mencionó GlobalExceptionHandler, ResponseEntity ni códigos 400/404/409" (IE 2.3.4, 30%)

Respuesta lista: "Cada microservicio tiene un `GlobalExceptionHandler` anotado con `@RestControllerAdvice`: los controllers no tienen try/catch, las excepciones burbujan y el handler las convierte en un `ResponseEntity` con código y body consistentes."



Fig. 5 — Ningún controller/service atrapa la excepción; burbujea hasta el handler global, que decide el código HTTP y arma el body consistente.

Excepción	HTTP	Cuándo ocurre (ejemplo real)
<code>NotFoundException</code> / <code>ResourceNotFoundException</code>	404	GET /api/espacios/99 con id inexistente
<code>ConflictException</code>	409	Registrar entrada con reserva no CONFIRMADA, o salida ya registrada
<code>BusinessException</code> (ms-pagos)	422	Crear un segundo cobro para el mismo acceso (viola el 1:1)
<code>MethodArgumentNotValidException</code> (Bean Validation)	400	POST sin campos obligatorios → body incluye el mapa de campos con el error por campo
<code>HttpMessageNotReadableException</code>	400	Body vacío o JSON mal formado
Excepción genérica	500	Cualquier error inesperado; se loguea con <code>log.error</code>

Honestidad técnica (te pueden pillar):

los handlers NO son 100% idénticos entre servicios — el profesor ya lo sospechó en EV2 (IE 2.3.1 al 60%: "falta confirmar homogeneidad"). Ejemplos reales: en **ms-accesos** el body es `{timestamp, status, mensaje}` y los errores de validación van en **"errores"**; en **ms-espacios** el body es `{timestamp, error, mensaje}` y la validación va en **"campos"**. Si te preguntan, di la verdad: "el patrón es el mismo (`@RestControllerAdvice` + `ResponseEntity` + códigos correctos), con pequeñas variaciones de nombres de campos entre servicios; una mejora pendiente sería unificar el DTO de error". Eso demuestra que conoces TU código de verdad.

4.2 ✖ "No desarrolló una entidad formal con PK, atributos y restricciones" (IE 2.1.3, 60%)

Entidad preparada: Acceso (ms-accesos, tabla **acceso** en db_accesos) — código real:

```
@Entity @Table(name = "acceso")
@Data @NoArgsConstructor @AllArgsConstructor // Lombok: getters/setters/constructores
public class Acceso {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id; // PK autoincremental

    @Column(nullable = false, name = "id_vehiculo_ref")
    private Long idVehiculo; // referencia LÓGICA a ms-vehiculos

    @Column(nullable = false, name = "id_espacio_ref")
    private Long idEspacio; // referencia lógica a ms-espacios

    @Column(nullable = true, name = "id_reserva_ref")
    private Long idReserva; // NULLABLE: acceso sin reserva permitido

    @Column(nullable = false) private String patenteEscaneada;
    @Column(nullable = false) private LocalDateTime fechaHoraEntrada;
    @Column(nullable = true) private LocalDateTime fechaHoraSalida; // null mientras está adentro
    @Column(nullable = true) private Integer minutos; // se calcula en la salida

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private EstadoEnum estado = EstadoEnum.ACTIVO; // ACTIVO | COMPLETADO (String en BD)
}
```

Guion de 30 segundos: "PK **id** autoincremental; restricciones NOT NULL en vehículo, espacio, patente, entrada y estado; **idReserva** nullable a propósito porque el negocio permite acceso sin reserva; el enum se persiste como String para legibilidad; y las columnas ***_ref** son referencias lógicas — no hay FK física porque cada servicio tiene su propia base. Aguas abajo, ms-pagos me referencia igual: **cobro.id_acceso_ref** con constraint UNIQUE (un cobro por acceso)."

4.3 ✖ "No interpretó un escenario de error concreto" (IE 2.3.3, 60%)

Escenario narrable #1 — servicio caído: "Si ms-espacios está caído y confirmo una reserva: **ReservaService.confirmar()** llama a **espacioClient.updateDisponibilidad()** → Feign no puede conectar y lanza **FeignException** → el service la captura y la traduce a una excepción de negocio → el handler responde con un código de error controlado, no un stacktrace crudo. En el log del servicio se ve el **Connection refused** hacia el puerto 8085, y en Eureka (:8761) se ve que la instancia desapareció del registro. Diagnóstico: revisar Eureka primero, levantar el servicio, reintentar."

Escenario narrable #2 — BD caída o puerto MySQL equivocado: "El servicio ni siquiera termina de arrancar: **Communications link failure** en el log al crear el pool de conexiones. Causa típica en nuestro entorno: XAMPP usa el puerto 3307 y las properties apuntan a otro — lo corregimos con **scripts\set-db-port.ps1**, que reescribe el puerto del

`spring.datasource.url` en los 10 servicios de una vez."

Escenario narrable #3 — 404 vs 409 en Newman: "Si un request de la colección falla con 409 al re-ejecutar, es un problema de idempotencia (dato duplicado de la corrida anterior); por eso la colección tiene scripts de limpieza pre-request."

4.4 ❌ "Aporte personal demasiado breve, sin commits ni archivos" (IE 2.5.3 — vale 5% mañana)

💜 Sol — guion de aporte personal

"Mis aportes verificables en el historial de commits:

- **Microservicios donde lidero commits** (según el análisis de autoría del repo): auth-service, user-service, ms-accesos, ms-reservas y ms-tarifas.
- **Scripts de reproducibilidad:** `manage.ps1` (dashboard TUI), `start-all.ps1` (arranque ordenado), `load-db.ps1` (carga las 10 BDs desde `db/00_run_all.sql`), `set-db-port.ps1`.
- **Pruebas:** suite de tests unitarios (ej. `AccesoServiceTest` con 7 casos) y colección Postman de 74 requests idempotente con captura automática del token JWT.
- **Documentación:** carpeta `docs/` (arquitectura, BD, seguridad, guías por servicio) y README de entrega con los 3 enlaces."

💙 Catalina — guion de aporte personal

"Mis aportes verificables:

- **ms-espacios** es el servicio donde tengo más commits directos (gestión de espacios y disponibilidad, el endpoint PUT `/disponibilidad` que consumen accesos y reservas).
- Contribuciones distribuidas en varios servicios (pagos, vehículos, tarifas) bajo mis identidades de git ("catalina" / "Catalina Aguirre") más el trabajo de sala compartido bajo "AO-Alumno".
- Trabajo funcional: pruebas manuales con Postman/Swagger, validación del flujo reserva→acceso→cobro, y revisión de la documentación."

Antes de mañana: corre `git log --author="catalina" --online` (y con tu otro correo) y memoriza 3 commits concretos con archivo y propósito — el profesor valora que nombres commits reales.

Tip para ambas:

el mapeo usuario-GitHub ↔ estudiante es uno de los 10 riesgos que el profesor anotó en la revisión grupal. Lleguen sabiendo qué identidad de git es de quién (incluida la aclaración de "AO-Alumno" = trabajo desde la sala).

4.5 ✖ "Sin modificación concreta con prueba antes/después"

(IE 2.4.4 en EV2 → mañana reaparece como IE 3.1.3/3.3.7)

Modificación ensayable en 2 minutos (practícala hoy): en el Gateway, cambiar el **Path** de ms-accesos de `/api/accesos/**` a `/api/v2/accesos/**` → reiniciar el gateway → mostrar que `GET localhost:8080/api/accesos` ahora da 404 y `/api/v2/accesos` responde 200 → revertir. Con eso demuestras: rutas, predicados, y prueba antes/después. Alternativa en código: cambiar el mensaje de una `ConflictException` y mostrar el cambio con Postman.

5. Levantar el sistema sola

IE 3.3.7 (10%) + 3.3.6 (6%)

5.1 Orden de arranque (memoriza — el profesor lo corrigió en el README)

1. **eureka-server** (8761) — el registro debe existir antes que nadie.
2. **Los 10 microservicios de negocio** — fase 3: auth, user, security, vehiculos, espacios, tarifas; fase 4: reservas, accesos; luego pagos y reportes (los que dependen de Feign al final).
3. **api-gateway** (8080) **AL FINAL** — para que al enrutar ya encuentre a todos registrados en Eureka.

```
# Opción A: script (arranca los 12 en orden, cada uno en su ventana)
.\scripts\start-all.ps1

# Opción B: manual, servicio por servicio
cd eureka-server; .\mvnw.cmd spring-boot:run
# ... esperar que arranque, luego los demás, gateway al final

# Opción C: JARs (como en el ZIP de entrega)
java -jar eureka-server.jar # → microservicios → gateway
```

5.2 Cómo verificar que todo está arriba

- **Eureka dashboard:** `http://localhost:8761` → deben aparecer las 11 instancias registradas.
- **Swagger directo:** `http://localhost:8085/swagger-ui/index.html` (o el puerto del servicio que sea).
- **Flujo por el Gateway:** login en `POST :8080/api/auth/login` (seed: `Test1234!`) → usar el token en `GET :8080/api/espacios`.
- **Newman (suite completa):** `newman run estacionamientos.postman_collection.json --env-var "base=http://localhost:8080"` → 74/74.

5.3 Tabla de diagnóstico rápido (lo que el profesor puede romper a propósito)

Síntoma	Causa probable	Arreglo
Port 8085 already in use	Instancia duplicada del servicio	Cerrar el proceso (o <code>manage.ps1</code> → stop) y relanzar; o cambiar <code>server.port</code> en properties
Communications link failure al arrancar	MySQL apagado o puerto equivocado (XAMPP usa 3307 en nuestro dev; los ZIP de entrega van en 3306)	Encender MySQL; alinear puerto con <code>scripts\set-db-port.ps1 -Port 3306</code>
Unknown database 'db_accesos'	Bases no creadas	<code>.\scripts\load-db.ps1</code> (ejecuta <code>db/00_run_all.sql</code> : crea las 10 BDs + seed)
Gateway responde 503 para un servicio	El servicio no está registrado en Eureka (caído o arrancando)	Mirar :8761; esperar el registro o relanzar el servicio

401 en todos los endpoints	Falta token o expiró (24 h)	Re-login en <code>/api/auth/login</code> y mandar Authorization: Bearer <token>
Schema-validation error de Hibernate	ddl-auto=validate y la tabla no coincide	Explicar: usamos validate en operación; para modificar en vivo se cambia a update

5.4 Despliegue: nativo vs Docker (IE 3.3.6)

	Nativo (ZIP 1)	Docker (ZIP 2)
Qué contiene	carpeta apps/ con los 12 JAR + arrancar-nativo.bat	apps/ + docker-compose.yml + arrancar-sistema.bat
Cómo arranca	El .bat lanza los JAR en orden: Eureka → servicios → Gateway	docker compose up : levanta MySQL, Eureka y los servicios como contenedores
Conexiones	localhost:3306 (MySQL), localhost:8761 (Eureka)	Nombres de servicio de la red de compose: mysql-db , eureka-server — dentro de un contenedor, localhost es el propio contenedor. Error clásico que el profesor documentó.
Datos iniciales	load-db.ps1 / SQL manual	init.sql montado como volumen — <i>solo corre la primera vez</i> (si el volumen ya existe, no se re-ejecuta)
Puerto ocupado	cambiar server.port	cambiar solo el puerto externo del mapeo en compose (" 8081:8080 ")

6. Banco de preguntas con respuestas modelo

Fuentes: los 10 "riesgos de defensa" que el profesor escribió en la revisión grupal EV2, las 6 preguntas EV2 de cada una, y los focos nuevos de la rúbrica EV3 (testing, Swagger, YAML, Docker).

6.1 Las 6 preguntas EV2 de Catalina (nunca las respondió por escrito — van con respuesta modelo)

P · 1. Arquitectura general: rol de Eureka, Gateway, Auth, User y 4 microservicios de dominio.

"El sistema tiene 12 servicios. **Eureka** (8761) es el registro: cada servicio se inscribe con su **spring.application.name** y así se descubren entre sí sin IPs fijas. El **API Gateway** (8080) es el punto único de entrada: enruta con **lb://** según el path y valida el JWT antes de dejar pasar. **auth-service** emite los tokens (login por email, BCrypt); **user-service** administra clientes, tipos y suscripciones. De dominio: **ms-espacios** (disponibilidad física), **ms-reservas** (máquina de estados de la reserva), **ms-accesos** (entrada/salida y minutos) y **ms-pagos** (fórmula de cobro con descuentos). Cada uno con su propia base MySQL — database per service."

P · 2. Flujo Controller → Service → Repository en ms-reservas, con DTOs.

"El **Controller** solo orquesta HTTP: recibe el JSON como **ReservaCreatedTO** (con Bean Validation), llama al service y devuelve **ReservaResponseDTO** en un **ResponseEntity** — nunca expone la entidad JPA como contrato. El **Service** (**@Service @Transactional**) concentra la lógica: validar cliente/vehículo/espacio por Feign, aplicar la máquina de estados, loguear con **@Slf4j**. El **Repository** (**JpaRepository<Reserva, Long>**) persiste. Las excepciones suben al **GlobalExceptionHandler**. Ventaja de los DTOs: desacoplan el contrato REST del modelo de datos — puedo cambiar la entidad sin romper a los consumidores."

P · 3. Crear una reserva: idCliente, idVehiculo, idEspacio, validación Feign, PENDIENTE, persistencia.

"**create()** recibe los 3 ids + fechas. Antes de guardar valida remotamente: cliente existe y **activo=true** (ClienteClient → user-service), vehículo activo (VehiculoClient), espacio activo y **disponible** (EspacioClient). Si algo falla, Feign lanza excepción (un 404 remoto llega como **FeignException**, no como null) y se traduce a NotFound/Conflict — la reserva NO se guarda. Si todo pasa, se persiste con **estado=PENDIENTE** y los ids como referencias lógicas Long. Importante: crear la reserva *no bloquea* el espacio todavía."

P · 4. Confirmar una reserva y qué pasa con la disponibilidad.

"**confirmar()** valida la transición (solo desde PENDIENTE), cambia a CONFIRMADA y ahí sí llama por Feign a `PUT /api/espacios/{id}/disponibilidad?disponible=false` para bloquear el espacio. Por eso **cancelar()** no toca ms-espacios: en PENDIENTE el espacio nunca se bloqueó. Y la liberación ocurre en ms-accesos al registrar la salida (`disponible=true` de vuelta)."

P · 5. ¿Por qué referencias lógicas Long en vez de relaciones JPA entre bases?

"Porque cada microservicio es dueño exclusivo de su base (10 BDs separadas). Una **@ManyToOne** entre servicios exigiría que ambas tablas vivan en el mismo esquema — eso acopla los despliegues y rompe la autonomía. Guardamos **Long** planos (`id_espacio_ref`) y garantizamos la integridad en la capa de servicio validando por Feign antes de persistir. El trade-off es consistencia eventual: lo aceptamos a cambio de servicios independientes. Dentro de un mismo servicio sí usamos JPA real: **Vehículo** → **TipoVehículo** es **@ManyToOne** porque ambas tablas están en `db_vehiculos`."

P · 6. Aporte personal.

Usar el guion personal de la sección 4.4 — con commits y archivos concretos, nunca "hice documentación y pruebas" a secas (eso costó 30% en EV2).

6.2 Los temas EV2 de Sol (ya respondidos bien — repaso exprés)

- **Gateway y lb://**: el Gateway centraliza la entrada; `lb://ms-pagos` = resolución por Eureka con balanceo, sin IPs hardcodeadas. (Sol sacó 100% aquí — mantener la explicación, agregando la palabra *predicates*.)
- **Flujo completo e información intercambiada**: ver sección 3.3 — Sol sacó 100%.
- **Fórmula de cobro**: sección 3.4 — 100% en EV2; ahora además puedes citar el test que la valida (5400.00 / 6840.00).
- **Scripts**: `load-db` (crea/recarga las 10 BDs), `set-db-port` (alineando el puerto MySQL en todas las properties), `start-all` (orden correcto), `manage` (TUI que integra todo). Reproducibilidad = cualquier persona clona y levanta igual.
- **Newman/idempotencia**: 74 requests contra el Gateway con token capturado en el login; idempotente gracias a scripts de limpieza `pre-request` → se puede re-ejecutar y siempre da 74/74, resultado trazable.

6.3 Preguntas nuevas estilo EV3 (testing, Swagger, YAML, Docker)

P · ¿Qué diferencia hay entre probar un Service y probar un Controller? (reflexión #3 de la guía)

"El Service test valida lógica de negocio con Mockito puro — mockeo Repository y Feign Clients, y afirmo sobre resultados y excepciones. El Controller test valida el contrato HTTP con `@WebMvcTest` + `MockMvc` — mockeo el Service y afirmo sobre ruta, código de estado y JSON (`jsonPath`). Errores que detecta el de Controller: ruta mal mapeada, código HTTP incorrecto, JSON con campos distintos, validación que no dispara. Errores que detecta el de Service: regla de negocio mal implementada, cálculo erróneo, excepción que no se lanza."

P · ¿Por qué las pruebas no deben usar la base de datos real? (reflexión #1)

"Tres razones: aislamiento (un test no debe depender del estado que dejó otro test o el desarrollo manual), velocidad y reproducibilidad (mismo resultado en cualquier máquina, incluso sin MySQL). Por eso los Service tests mockean el Repository, y los tests de integración usan H2 en memoria con `ddl-auto=create-drop` en `application-test.properties`, con Eureka deshabilitado."

P · ¿Qué hace exactamente `verify()` y cuándo es imprescindible?

"Comprueba interacciones, no valores: que el service llamó (o no llamó) a un método del mock. Es imprescindible cuando el efecto no está en el valor de retorno: en `registrarSalida`, el retorno es el DTO, pero la liberación del espacio es una llamada Feign — solo `verify(espacioClient).updateDisponibilidad(1L, true)` puede probarla. Y `verify(repo, never()).save(any())` prueba que ante un error NO se persistió nada."

P · Muéstrame en Swagger cómo se consume un endpoint.

Abrir :8085/swagger-ui/index.html → `POST /api/espacios` → mostrar el schema de `EspacioCreateDTO` (campos y obligatorios) → "Try it out" con un body válido → 201 y el response DTO → repetir con body inválido → 400 con el mapa `campos`. Cerrar con: "la doc se genera del código con springdoc, por eso nunca se desalinea del comportamiento real".

P · ¿Qué pasa si corro los tests y aparece `No qualifying bean of type EspacioService?`

"Es el error clásico de `@WebMvcTest` sin mock: ese slice solo levanta la capa web, no los services reales. El Controller necesita su dependencia, así que hay que declararla como `@MockitoBean`. Está en la sección 11.6 de la guía de testing."

P · ¿Por qué en Docker el servicio no encuentra a MySQL si en local funciona?

"Porque dentro de un contenedor `localhost` apunta al propio contenedor. En la red de docker compose los servicios se resuelven por nombre: la URL pasa de `jdbc:mysql://localhost:3306/db_accesos` a `jdbc:mysql://mysql-db:3306/db_accesos`, y Eureka de `localhost:8761` a `eureka-server:8761`. Es el error #1 de la guía Docker del profesor."

P · ¿Cómo garantizan que un acceso tenga máximo un cobro?

"Doble capa: constraint UNIQUE en `cobro.id_acceso_ref` (BD) y validación en `PagoService.crear()` que consulta `findByIdAcceso` y lanza `BusinessException` ('Ya existe un cobro') → 422. Hay un test unitario que lo prueba con el repo mockeado."

P · ¿Por qué usan @PutMapping para cancelar/confirmar en vez de @PatchMapping?

"Decisión técnica documentada: el cliente HTTP por defecto de Feign (`URLConnection`) no soporta el verbo PATCH, así que todas las actualizaciones parciales entre servicios son PUT. Además fue el patrón enseñado en clases."

6.4 Los 10 riesgos que el profesor dejó anotados (revisión grupal EV2, sección E)

1. Levantar Eureka, Gateway y microservicios en orden → sección 5.1 ✓
2. Ejecutar/explicar la colección de 74 requests → sección 6.2 ✓
3. Cómo se carga `db/00_run_all.sql` y qué bases crea → `load-db.ps1`, 10 BDs (01_db_auth ... 09_db_pagos) + seed (2 clientes, tipos, tarifas, bancos; password seed `Test1234!`) ✓
4. Validación JWT en el Gateway → `JwtAuthFilter`: extrae el Bearer, valida firma con el secret compartido, expiración 24 h; rutas públicas `/api/auth/**` y Swagger; sesiones STATELESS ✓
5. Flujo reserva → entrada → salida → cobro → reporte → sección 3.3 ✓
6. Justificar referencias lógicas Long → pregunta 5 de Catalina (6.1) ✓
7. Errores Feign cuando cliente/vehículo/espacio no existen → `FeignException` capturada y traducida; escenario narrado 4.3 ✓
8. Control de disponibilidad al confirmar y al salir → confirmar bloquea, salida libera (6.1 #4) ✓
9. Mapear usuarios GitHub ↔ Catalina/Sol → tip de la sección 4.4 ✓
10. Evidencia de planificación → Trello ya existe para EV3: tablero "DSY1103-G7 Estacionamiento Inteligente" con 48 tarjetas en listas EV1/EV2/EV3 (en EV2 esto valió 0 por no evidenciarse — ahora sí, menciónalo proactivamente) ✓

7. Checklist de la noche anterior

- ☐ **Escribir de memoria** la plantilla de test de la sección 2.6 (en papel, sin mirar) — dos veces: caso de error y caso feliz con **verify**.
- ☐ Correr `.\mvnw.cmd test` en ms-accesos y ms-reservas para confirmar que todo está verde y *ver* el output que verás mañana.
- ☐ Levantar el sistema completo una vez con **start-all.ps1**, abrir Eureka (:8761), un Swagger (:8085) y hacer login + un GET por el Gateway.
- ☐ Practicar la modificación antes/después (ruta del Gateway, sección 4.5) con cronómetro: < 3 minutos.
- ☐ Recitar la fórmula de cobro y el ejemplo 10000 → 6840 con descuentos en cascada.
- ☐ Recitar la entidad **Acceso** con PK, NOT NULLs, el nullable de idReserva y el enum String.
- ☐ Recitar la tabla excepción → código HTTP (404/409/422/400/500) y decir "@RestControllerAdvice" y "ResponseEntity" al menos una vez en la defensa.
- ☐ Preparar el guion de aporte personal (sección 4.4) con 3 commits reales: **git log --author= ... --oneline | head**.
- ☐ Saber el orden de arranque al derecho y al revés: Eureka → microservicios → Gateway (y por qué el Gateway va al final).
- ☐ Dormir. 15 minutos de defensa se ganan con claridad, no con trasnoche.

Frase de cierre si te quedas en blanco:

vuelve siempre al flujo — "el Controller recibe, el Service decide, el Repository persiste, Feign comunica, el handler traduce errores a códigos HTTP, y el test lo demuestra con mocks". Desde esa frase puedes reconstruir cualquier respuesta del ramo.